# A Closed-Loop Day-2 Remediation Architecture for O-RAN Cloud Infrastructure Using Standardized Interfaces

David Kypuros
Red Hat
dkypuros@redhat.com

31 May 2026

## Abstract

This paper describes a closed-loop Day-2 remediation pattern for O-RAN Cloud RAN deployments. The pattern composes four published interface contracts: the O-RAN O2 Infrastructure Management Services (O2 IMS) API as the SMO-to-O-Cloud provisioning surface; Kubernetes-native Custom Resource Definitions (Metal3 BareMetalHost / HostFirmwareComponents, Machine Config Operator, Kernel Module Management) as the delivery substrate; TM Forum TMF688 and TMF921 as the audit and intent envelopes; and IEEE 1588 / O-RAN WG6 Cloud Notifications as the timing-domain observation contract. We claim the novelty lies in the conjunction of O2 General Aspects and Principles, O2 IMS Interface, Metal3, and Machine Config Operator under a single deterministic routing rule that classifies infrastructure faults against the O-RAN WG6 O-Cloud resource model before any large language model is consulted. The contribution extends a prior multivendor diagnostic substrate by closing the loop from fault to audited remediation through a five-gate guardrail and a digital twin pass. The runtime is open-source and shipped with a verify gate that produces a binary pass/fail any reviewer can reproduce. We acknowledge that v0 runtime is stub-driven and that production wiring is future work.

## 1 Introduction

Day-2 operations on a multivendor Cloud RAN deployment span a wide functional surface: timing drift on a worker node, firmware regressions on accelerator NICs, host service interference with the PTP hardware clock, driver swaps for fixed-function offloads, configuration churn from upstream SMO intents. Each of these is observable, each has an audited path to remediation, and yet the bridges between diagnosis and execution are uneven across vendors and standards bodies.

Prior agentic-AI work for RAN automation has clustered around two failure modes. The first stops at the SMO boundary: an agent converges on a root cause and emits an intent, but the intent never crosses into a concrete delivery contract. The second invents a parallel controller plane that side-steps O-RAN's own resource layering, which is operationally fragile and politically expensive in a multivendor ecosystem. A prior multivendor diagnostic publication established a fault-localization substrate for Cloud RAN troubleshooting [8] but left the loop open: the diagnostic agents converge on a root cause and emit an artifact, while the actual remediation path remains undefined.

This paper closes the loop. We describe a harness pattern that takes the diagnostic artifact as input, classifies it against a deterministic taxonomy grounded in the O-RAN O-Cloud resource model [18, 24], routes the resulting proposal through the O-RAN O2 IMS interface [25] into Kubernetes-native CRD contracts (Metal3 [15] and the Machine Config Operator [28], with the Kernel Module Management Operator [9] as a third delivery path), and emits a TM Forum TMF688 audit envelope [31] with an embedded reversibility profile before any apply is authorized. The four pieces (observation loop, cognitive middle, routing decision, sandbox plus operator seam) are described in order. The architectural novelty is the simultaneous conjunction of references [24], [25], [15], and [28], which we will refer to as the 2+3+8+9 conjunction. The numbers 2, 3, 8, 9 follow the AMA-numbered bibliography in the source repository's `docs/references.md`, where the four cited works are entries 2 (O2 GA&P), 3 (O2 IMS Interface), 8 (Metal3 BMO), and 9 (MCO). The rendered bibliography at the end of this paper, ordered alphabetically by citation key under the `plain` bibliography style, will assign different numbers to the same four works. KMM (Kernel Module Management, [9]) is a third Kubernetes-native delivery path within the same down-route but is not a member of the four-element conjunction.

The remainder of the paper is organized as follows. Section 2 establishes the standards and project context for each interface composed. Section 3 describes the architecture in three logical pieces. Section 4 summarizes the prototype, the five committed walkthrough scenarios, and the verify gate. Section 5 discusses positioning against adjacent projects and the standards-track ask. Section 6 concludes.

## 2 Background

**O-RAN architecture and the O-Cloud surface.** The O-RAN Alliance Working Group 1 architecture description [18] and Working Group 6 O2 General Aspects

and Principles [24] jointly define the O-Cloud resource model and the SMO-to-O-Cloud interface family. The O2 IMS specification [25] carries the infrastructure provisioning request and response envelopes; O2 DMS [23] carries deployment management. This work targets O2 IMS exclusively: the harness handles infrastructure remediation, not deployment lifecycle. The reference O2 IMS implementation cited throughout is Red Hat's open-source O-Cloud Manager [27]. The Decoupled SMO Architecture technical report [20] introduces the SMO Service (SMOS) abstraction, into which a closed-loop remediation harness fits naturally as a candidate first-class SMOS.

**Delivery substrate.** Below the O2 IMS interface, three Kubernetes-native operators carry the actual change: the Machine Config Operator [28] for host configuration, the Kernel Module Management Operator [9] for driver swaps, and Metal3 [15, 16] for bare-metal firmware and provisioning. Out-of-band firmware writes pass through the host BMC via DMTF Redfish [4].

**Timing and observation.** The PTP stack on each worker uses linuxptp [13] to discipline the PTP Hardware Clock (PHC) against an upstream Grandmaster, conforming to IEEE 1588-2019 [6] and the ITU-T G.8275.1 telecom profile [7]. The Red Hat PTP Operator [5] orchestrates linuxptp on the cluster, and the cloud-event-proxy sidecar [29] publishes state transitions as CloudEvents [3] carrying the O-RAN WG6 Cloud Notifications payload [19]. The harness consumes that notification envelope as the inbound fault contract.

**Audit and intent.** TM Forum TMF688 [31] provides the event-management envelope used as the audit sink. TMF921 [32] provides the intent management envelope used for the dual-route notification path from the harness to the partner SMO. The SID information framework [30] grounds the conceptual model that maps O-RAN resource classes to TM Forum entity types.

# 3 System Architecture

The architecture has three logical pieces and an overhead control: the left loop (observation), the cognitive middle, the right loop (routing and delivery), and the Killswitch as a global override. Figure 1 shows the macro layout.

## 3.1 Left loop: observation

Telemetry originates in the timing domain. Each Cloud RAN worker runs a PTP Hardware Clock disciplined by linuxptp daemons (`ptp4l` and `phc2sys`) [13], conforming to IEEE 1588-2019 [6] and the G.8275.1 telecom profile [7]. The PTP Operator [5] runs the daemons and the cloud-event-proxy sidecar [29] watches their state transitions. When the daemon state crosses a threshold (SYNCHRONIZED to HOLDOVER, master offset spike, monotonic PHC drift), the sidecar emits a CloudEvent [3] with the O-RAN WG6 Cloud Notifications payload [19].

The harness consumes this envelope as a `FaultPayload` (a JSON Schema draft-07 contract carried at `harness/schemas/FaultPayload.json`). The repository's scenario `D_phc_drift_hw_only` demonstrates the canonical divergence pattern that gives the harness its diagnostic edge: `ptp4l` reports SYNCHRONIZED (software view of the timing chain is healthy) while `phc2sys` reports monotonic frequency drift in parts-per-billion and the NIC reports rising `tx_hwtstamp_timeouts` (hardware view is not healthy). A purely software-layer observer would miss this; the harness reads both.

## 3.2 Cognitive middle

The Agentic Gateway terminates the CloudEvents stream and presents vendor-private tooling to the agent runtime under controlled MCP interfaces [1, 2, 14]. Four FastMCP servers run behind the gateway: `mcp-platform` (linuxptp daemons, host services, driver versions, NIC counters), `mcp-ran` (cell sync, PM counters, RAN parameters), `mcp-hardware` (NIC PHC introspection, BMC Redfish, CPU performance counters), and `mcp-ocloud` (the O-Cloud Manager inventory and alarms). The architectural pattern describes mTLS, OAuth2, and IP allowlist as the production posture; the v0 lab trusts localhost.

Three domain agents (Platform, RAN, Hardware) consume the MCP surface and converge on a root cause artifact (RCA). The cognitive middle's deterministic core is the Remediation Router. The router consults a 20-entry taxonomy declared in `harness/taxonomy.yaml` that maps every fault class onto the O-RAN WG6 O-Cloud resource model. The taxonomy carries an explicit grounding disclaimer in its first eighteen lines: the 20 subtypes are interpretive mappings within the three normative resource classes from O2-GAnP Figure 3.4-1 (Physical Resource, Logical Resource, Managed O-Cloud Service), not extensions to the spec.

The deterministic taxonomy lookup fires first; an LLM is consulted only when the result is the `ambiguous` class. This ordering is enforced in code and proven by the test suite. The repository's `contribution-1-routing-rule.yaml` declares the rule, the runtime at `harness/runtime/router.py` implements it, and the test `test_route_ambiguous_with_llm_mode_live` demonstrates that the LLM seam is wired through without becoming load-bearing on the routing decision.

## 3.3 Right loop: routing and dual-route delivery

The routing decision splits along the O-RAN resource-layer boundary. **Service-layer** faults (cell re-home, RAN parameter changes, slice intent updates, vDU software version rollouts) belong to the partner SMO. The harness does not execute them; it emits a TMF921 intent [32] and observes the dispatch result. This is the up route.

**Infrastructure-layer** faults go down through the O2 IMS interface [24, 25] into the O-Cloud Manager [27], which dispatches into one of three reconcilers: Machine Config Operator [28] for host configuration changes,

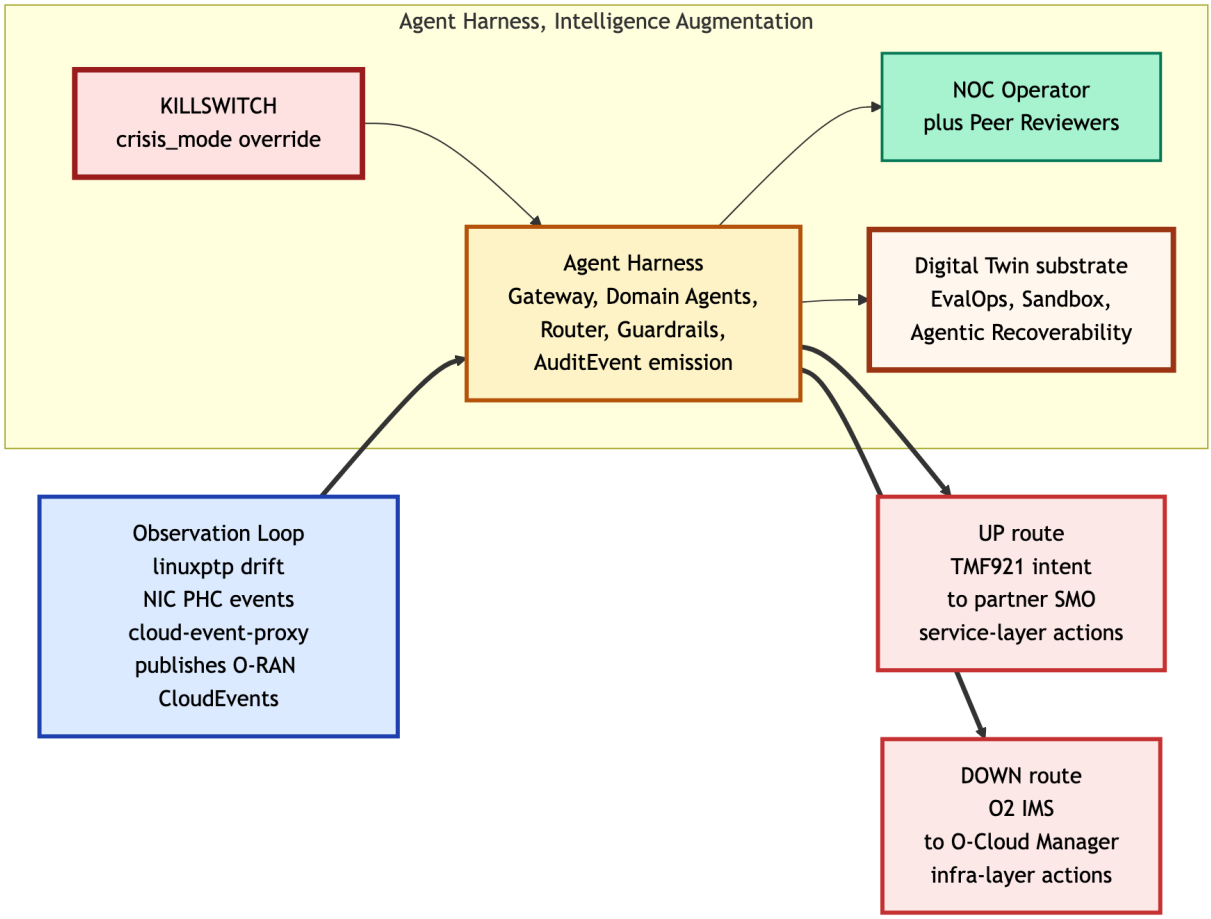


Figure 1: Seven-component macro architecture. The left side carries the observation loop (PTP Operator, cloud-event-proxy, O-RAN WG6 Cloud Notifications). The center carries the Agent Harness (Agentic Gateway plus four MCP servers plus three domain agents plus the deterministic Router). The right side forks: UP via TMF921 to the partner SMO, DOWN via O2 IMS to the O-Cloud. The Digital Twin substrate sits beneath the harness; the Killswitch is a global override above it.

KMM [9] for driver swaps, Metal3 BMO [15] for node firmware. Firmware writes pass through the host BMC via DMTF Redfish UpdateService.SimpleUpdate [4]. This is the down route. In the v0 implementation, the down route is a real Python function call from the router into a stubbed O-Cloud Manager that returns a shape-correct `o2ims_dispatch` envelope; the envelope is attached to the audit so the operator sees the O2 IMS hop as a row in the record, not as a synthesized path string.

The dual-route pattern fires both branches simultaneously when an infrastructure action carries inherent service-layer impact. Scenario `E_nic_firmware_update` is the teaching case: a NIC firmware update via Metal3 takes the host offline for a reboot window, so the harness emits a TMF921 companion intent up to the partner SMO in parallel with the O2 IMS down-route apply. The partner SMO's response (acceptance or rejection) lands on the audit's `dispatch_result` field before operator co-authorization. Scenario `E_with_smo_reject` demonstrates the rejection path: same down-route verdict, SMO rejects the maintenance window because neighboring cells are at 92 percent capacity. The operator sees both outcomes and is the reconciliation point.

### 3.4 Sandbox gate and reversibility

Before any apply, the proposal traverses a deterministic five-gate guardrail engine declared at `harness/guardrails.yaml` and executed at `harness/runtime/guardrail.py`. The gates fire in order: crisis_mode global override, sandbox apply_allowed (verdict produced by the Twin pass), action_allowlist, blast_radius caps (one node, one site, four cells in v0), and requiresHumanApproval. The Sandbox stage is operationally flexible: a same-topology mirrored cluster or a simulated linuxptp plus NIC driver stack satisfies the contract.

Every audit event carries a populated `ReversibilityProfile` with seven fields: rollback intent, blast-radius-if-reverse, rebuild timeline, replication history, validation history, risk profile burn-down, and confidence in reversibility. The intent is that the operator has the rollback contract at decision time, not after-the-fact. Figure 2 shows the governance layer in detail.

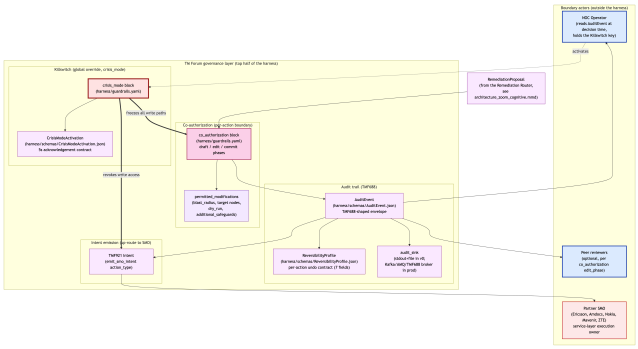

Figure 2: Governance layer zoom. The TMF688 audit event carries the proposal, the guardrail result, the sandbox verdict, the optional companion intent with its dispatch result, and the seven-field ReversibilityProfile. The co-authorization contract sits between draft and commit. The Killswitch (crisis_mode) is the global override.

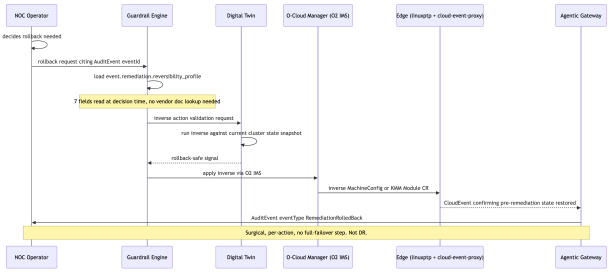


Figure 3: Sequence diagram for a single closed-loop remediation cycle. PTP CloudEvent arrives at the gateway, traverses the cognitive middle to a RemediationProposal, passes the sandbox apply-allowed gate and the five-gate guardrail, lands on the operator's co-authorization seat, and emits a TMF688 audit event with the reversibility profile attached.

### 3.5 Killswitch as global override

A `crisis_mode` flag in `guardrails.yaml` is the global override. When activated by a NOC supervisor in response to a macro event (storms, security incidents, regional outages), all write paths revoke, EvalOps confidence pins to zero, and telemetry routes to a manual queue. The Reversibility Profile rewinds one applied action; the Killswitch freezes all of them. The two compose.

## 4 Prototype and Walkthroughs

The repository ships five end-to-end fixture scenarios under `scenarios/`: `A_fw_lldp_agent` (a host service interfering with PTP, routed to MCO), `A_prime_ice_driver` (an outdated NIC driver causing PHC drift, routed to KMM), `D_phc_drift_hw_only` (the hardware-only divergence demonstrating the diagnostic edge), `E_nic_firmware_update` (the dual-route firmware case with accepted companion intent), and `E_with_smo_reject` (the same firmware path with a rejected companion intent). Each scenario carries a `fault_payload.json` input, a remediation YAML, and a committed `audit_event.json` fixture that the verify gate replays against the actual walker output.

The runtime pipeline (walker, router, sandbox, guardrail) is deterministic. A reviewer can clone the repository, install only PyYAML and jsonschema, and run `python3 scripts/verify.py`. The gate completes ten checks in under five seconds: citation header completeness on every authored YAML and JSON, JSON and YAML parse, conformance-table bidirectional completeness, an em-dash audit, a leakage guard, README structural sanity, file counts, JSON Schema validation of every scenario fault payload and audit event (all five scenarios), and end-to-end walker replay against committed audit fixtures on eight material fields (seven named fields plus the reversibility profile confidence sub-field). The reference state in the repository at the time of writing reports ten of ten checks pass.

## 5 Discussion

**The 2+3+8+9 novelty conjunction.** The architecture's central claim is that a single `RemediationProposal` travels through the O2 IMS contract surface [24, 25] and lands at a Kubernetes-native CRD reconciler [15, 28] without a custom controller plane in between. The conjunction of these four references is not a re-derivation of prior architectures; existing agentic-RAN work either stops at the SMO boundary without a delivery story, or proposes a parallel controller plane. The harness pattern composes existing contracts.

**Comparison to ONAP, Nephio, OSM.** ONAP [11], Nephio [10], and Open Source MANO [12] are orchestration platforms that manage network function lifecycle at scale. The harness pattern is closed-loop remediation: fault-driven cognition that emits a proposal, gates it, audits it, and observes the operator's commit. The two compose. Nephio can deliver the standing topology; the harness handles post-deployment Day-2 anomalies on top of that topology.

**Positioning against the Decoupled SMO Architecture.** The closed-loop harness fits the SMOS abstraction described in WG1 TR R004 [20]. It exposes a TMF688 audit surface and a TMF921 intent surface, both SMOS-compatible by interface shape (interop testing against an SMOS-conformant catalogue is a v1 work item). A reasonable standards-track ask is for an nGRG output that names a Closed-Loop Remediation SMOS as a first-class element of the Decoupled SMO catalogue, and a TM Forum Implementation Guide companion that documents two harness-unique extensions as candidate API additions: a `dispatch_result` field on the TMF921 companion intent envelope, and the seven-field ReversibilityProfile attached to every TMF688 audit event.

**Layer-boundary discipline.** The harness is explicit about what it does not target. O-RAN R1 [26] is the SMO-internal service-exposure interface for rApps; the harness's MCP gateway is internal agent scaffolding, not R1. A1 [17] targets Near-RT RIC behavior (slice SLA,

traffic steering); the harness targets the infrastructure layer below. E2 [21] governs Near-RT RIC interactions with O-CU and O-DU; the harness does not consume E2 directly. O1 [22] governs SMO-to-managed-element OAM; the harness operates below the managed-element boundary because the O-Cloud is platform, not managed element. These disclaimers are not modesty; they preserve the composability claim.

**Honest-scope acknowledgements.** The v0 runtime is stub-driven for the five committed scenarios. The platform stubs at `5G_O-RAN_SIM/oam/` emit shape-correct envelopes without crossing real network boundaries. The five-gate guardrail enforces v0 absolute limits (one node, one site, four cells); a production deployment would wire to a policy-as-code system (OPA or Kyverno) and source limits from a service catalog. Multi-cluster scale is not demonstrated. The co-authorization edit-phase (operator amending the proposal before commit instead of approving or rejecting it) is the v1 target; v0 enforces a boolean approval gate.

# 6 Conclusion

This paper has described a closed-loop Day-2 remediation harness pattern for O-RAN Cloud RAN deployments. The pattern composes four pre-existing standardized contracts (O-RAN O2 IMS, Kubernetes-native CRDs, TM Forum TMF688 and TMF921, IEEE 1588 plus O-RAN Cloud Notifications) under a deterministic routing rule grounded in the O-RAN WG6 O-Cloud resource model. The 2+3+8+9 conjunction is the architectural novelty. The runtime is open-source, exercised end-to-end by a fixture-driven verify gate, and ships with five walkthrough scenarios including a dual-route firmware case with both accepted and SMO-rejected variants. v0 limits are declared in code and in this paper.

# Acknowledgements

This work extends a prior multivendor diagnostic substrate published as an Ericsson Technology Blog and co-authored with S. Kiani Mehr, N. Korati Prasanna, M. Vazquez Cebrian, S. Srikanthan, and P. Venkatesh [8]. Killswitch design feedback was received from AT&T operations stakeholders. Apache-2.0 licensed source code: https://github.com/dkypuros/oran-agent-harness.

# References

[1] Anthropic. Model context protocol python SDK. https://github.com/modelcontextprotocol/python-sdk, 2026. Accessed 14 May 2026.

[2] Anthropic. Model context protocol specification. https://modelcontextprotocol.io, 2026. Accessed 14 May 2026.

[3] Cloud Native Computing Foundation. CloudEvents version 1.0 specification: a vendor-neutral specification for describing event data. https://cloudevents.io, 2026. Accessed 14 May 2026.

[4] Distributed Management Task Force. Redfish specification (DMTF DSP0266). https://www.dmtf.org/standards/redfish, 2026. Out-of-band management of servers and infrastructure. Accessed 14 May 2026.

[5] Red Hat. PTP operator for OpenShift documentation. https://docs.redhat.com/en/documentation/openshift_container_platform, 2026. Accessed 14 May 2026.

[6] Institute of Electrical and Electronics Engineers. IEEE Std 1588-2019: IEEE standard for a precision clock synchronization protocol for networked measurement and control systems. https://standards.ieee.org/ieee/1588/6825/, 2019.

[7] International Telecommunication Union, Telecommunication Standardization Sector. ITU-T G.8275.1: Precision time protocol telecom profile for phase / time synchronization with full timing support from the network. https://www.itu.int/rec/T-REC-G.8275.1, 2022.

[8] S. Kiani Mehr, N. Korati Prasanna, D. Kypuros, M. Vazquez Cebrian, S. Srikanthan, and P. Venkatesh. Multivendor agentic AI solution for cloud RAN troubleshooting. Ericsson Technology Blog. https://www.ericsson.com/en/blog/2026/5/multivendor-agentic-ai-solution-for-cloud-ran, 2026. Published 11 May 2026. Accessed 14 May 2026.

[9] Kubernetes Special Interest Groups. Kernel module management operator (KMM). https://github.com/kubernetes-sigs/kernel-module-management, 2026. Accessed 14 May 2026.

[10] Linux Foundation Networking. Nephio: Cloud-native network automation. https://nephio.org, 2026. Accessed 14 May 2026.

[11] Linux Foundation Networking. Open network automation platform (ONAP). https://www.onap.org, 2026. Accessed 14 May 2026.

[12] Linux Foundation Networking. Open source MANO (OSM). https://osm.etsi.org, 2026. Accessed 14 May 2026.

[13] linuxptp project. ptp4l(8) manual page: PTP boundary / ordinary clock daemon for Linux. https://linuxptp.sourceforge.net, 2026. Accessed 14 May 2026.

[14] J. Lowin. FastMCP: a pythonic framework for building model context protocol servers and clients. https://github.com/jlowin/fastmcp, 2026. Accessed 14 May 2026.

[15] Metal3 project. Baremetal operator: BareMetalHost, HostFirmwareSettings, HostFirmwareComponents Custom Resource Definitions. https://github.com/metal3-io/baremetal-operator, 2026. Accessed 14 May 2026.

[16] Metal3 project. Metal3: Bare metal host provisioning for Kubernetes. https://metal3.io, 2026. Accessed 14 May 2026.

[17] O-RAN Alliance. O-RAN A1 interface: General aspects and principles and application protocol, working group 2 (O-RAN.WG2.TS.A1GAP-R005-v05.03 and O-RAN.WG2.TS.A1AP-R005-v06.00). https://www.o-ran.org/specifications, 2026. Accessed 30 May 2026.

[18] O-RAN Alliance. O-RAN architecture description, working group 1 specification. https://www.o-ran.org/specifications, 2026. Accessed 14 May 2026.

[19] O-RAN Alliance. O-RAN cloud notifications specification, working group 6 (O-RAN.WG6.Cloud-Notifications). https://www.o-ran.org/specifications, 2026. Accessed 14 May 2026.

[20] O-RAN Alliance. O-RAN decoupled SMO architecture, working group 1 technical report (O-RAN.WG1.TR.Decoupled-SMO-Architecture-R004-v03.00). https://www.o-ran.org/specifications, 2026. Accessed 30 May 2026. Source for SMOS terminology.

[21] O-RAN Alliance. O-RAN E2 interface family: General aspects and principles, application protocol, and service models, working group 3 (E2GAP, E2AP, E2SM-KPM, E2SM-RC, E2SM-LLC). https://www.o-ran.org/specifications, 2026. Accessed 30 May 2026.

[22] O-RAN Alliance. O-RAN O1 interface specification, working group 10 (O-RAN.WG10.TS.O1-Interface.0-R005-v18.00). https://www.o-ran.org/specifications, 2026. Accessed 30 May 2026.

[23] O-RAN Alliance. O-RAN O2 DMS interface specification (deployment management services), O-RAN.WG6.O2DMS-Interface. https://www.o-ran.org/specifications, 2026. Working Group 6 specification. Accessed 14 May 2026.

[24] O-RAN Alliance. O-RAN O2 general aspects and principles, version 01.02 (O-RAN.WG6.O2-GAnP-v01.02). https://www.o-ran.org/specifications, 2026. Working Group 6 specification, with successor in R005 release train (O-RAN.WG6.TS.O2-GA&P-R005-v10.00). Accessed 14 May 2026.

[25] O-RAN Alliance. O-RAN O2 IMS interface specification (infrastructure management services), O-RAN.WG6.O2IMS-Interface. https://www.o-ran.org/specifications, 2026. Working Group 6 specification. Accessed 14 May 2026.

[26] O-RAN Alliance. O-RAN R1 interface: General aspects and principles and application protocols for R1 services, working group 2 (O-RAN.WG2.TS.R1GAP-R005-v13.00 and O-RAN.WG2.TS.R1AP-R005-v10.00). https://www.o-ran.org/specifications, 2026. Accessed 30 May 2026.

[27] OpenShift KNI. oran-o2ims: O-Cloud manager. https://github.com/openshift-kni/oran-o2ims, 2026. Accessed 14 May 2026.

[28] OpenShift project. Machine config operator: declarative configuration of Red Hat Enterprise Linux CoreOS. https://github.com/openshift/machine-config-operator, 2026. Accessed 14 May 2026.

[29] Red Hat Cloud Native Events project. cloud-event-proxy: PTP notification publisher emitting O-RAN-compliant CloudEvents from linuxptp state changes. https://github.com/redhat-cne/cloud-event-proxy, 2026. Accessed 14 May 2026.

[30] TM Forum. GB922 information framework (SID) concepts and principles. https://www.tmforum.org, 2026. Accessed 14 May 2026.

[31] TM Forum. TMF688 event management API user guide and REST specification. https://www.tmforum.org/oda/open-apis/table/tmf688, 2026. Accessed 14 May 2026.

[32] TM Forum. TMF921 intent management API user guide and REST specification. https://www.tmforum.org/oda/open-apis/table/tmf921, 2026. Accessed 14 May 2026.